SAE 1.01 Implémentation d'un besoin client

BUT Informatique – 1A

Alexandre CHENAL,

Kajan ARULNATHAN,

Mikael GALTIER,

Ariles DADI,

Ryan BACHATENE,

Zayd CROMBEZ.

UPEC
Connaissance - Action
UNIVERSITÉ
PARIS-EST CRÉTEIL
VAL DE MARNE

SOMMAIRE :

1. Besoins exprimés par le client
2. Organisation des tâches
3. Environnement de développement
4. Architecture du projet
5. Implémentation des fonctionnalités
6. Interface utilisateur
7. Tests et validation des fonctionnalités
8. Résultats et Analyse

1/Introduction :

Dans le cadre de la SAÉ 1.01, notre projet consiste à répondre à exploiter et analyser les données de l'Assurance Maladie de manière interactive via une application web. Cette application permet de consulter des données contenues dans une base de données relationnelle, ainsi que leur analyse statistique et leur visualisation sous forme de graphiques. L'Assurance Maladie offre au grand public un vaste ensemble d'informations variées et détaillées sur les professions de santé exerçant en libéral. Ces données couvrent une trentaine de métiers, incluant les médecins (généralistes et spécialistes), les chirurgiens-dentistes, les sage-femmes, ainsi que les auxiliaires médicaux tels que les infirmiers, les masseurs-kinésithérapeutes, les orthophonistes, les orthoptistes et les pédicures-podologues.

2/Organisation des tâches :

Pour mener à bien ce projet, notre équipe a réparti les responsabilités afin de maximiser l'efficacité et de tirer parti des compétences de chacun. Chaque membre a contribué à des aspects spécifiques du développement, garantissant une progression cohérente et structurée du projet. Dans notre équipe certains ce sont occupés de réaliser l'interface graphique de l'application web (« frontend ») et d'autres ont réalisés la mise en relation entre l'application web et la base de données (la partie « backend »).

3/Environnement de développement :

Afin de mener à bien notre projet, nous avons utilisés des outils possédants des technologies efficace et une interface utilisateur pratique.

L'environnement de développement (IDE) que nous avons choisi est Visual Studio Code, un IDE connu et efficace.

En ce qui concerne la partie backend de notre application, nous avons utilisé le micro-Framework web : Flask. Il a été utilisé pour gérer les routes, traiter les requêtes des utilisateurs, mettre en relation la base de données SQLite avec l'application et afficher les pages web.

Troisièmement, pour la base de données nous avons utilisés SQLite sur le logiciel Dbeaver. SQLite est très utile pour son intégration en Python.

Pour analyser les données en python il a fallu utiliser certaines bibliothèques tels que Pandas pour intégrer et manipuler les données importées en SQLite en créant des DataFrames pour crée des tables similaires à une table dans une base de données.

Pour la création de graphique et afin d'obtenir des analyses de manières visuelle, nous avons utilisé Matplotlib. Cette bibliothèque permet de crée des graphiques tels que des diagrammes, histogrammes etc. Ces graphiques sont aussi largement personnalisables.

4/Architecture du projet :

Le projet est organisé de la même manière que le projet « flask-mini » mis à disposition sur Eprel. Le projet est constitué de quatre dossiers. Premièrement le dossier « model » contenant le fichier « db.py » où se trouve principalement les requêtes SQLite et l'importation de la base de données comme montré ci-dessous :

Cette partie du code sert à importer la base de données dans le fichier.

```
DATABASE = "debase_db.sqlite3"
db_path = os.path.join(os.path.dirname(__file__), 'data', DATABASE)
```

Le code ci-dessous sert à ouvrir et fermer la base de données et d'écrire n'importe quelle requête SQLite afin d'obtenir des informations dans la base de données.

```
def montant_moyen_prescriptions(annee=None):
    conn = connect_db()
    query = """
    SELECT libelle_poste_prescription, AVG(montant_moyen_prescription_integer) AS montant_moyen, Annee
    FROM Prescriptions
    """
    if annee:
        query += f" WHERE Annee = {annee}"

    query += " GROUP BY libelle_poste_prescription;"

    result = pd.read_sql_query(query, conn)
    conn.close()
    return result
```

Par la suite il y a les dossiers contenant les pages HTML que l'on appelle « templates ». Ces pages sont stylisées grâce à un seul fichier CSS qui se trouve dans le dossier « static ». A l'aide du Framework Flask, nous pouvons obtenir le CSS grâce à ce code intégré au langage HTML :

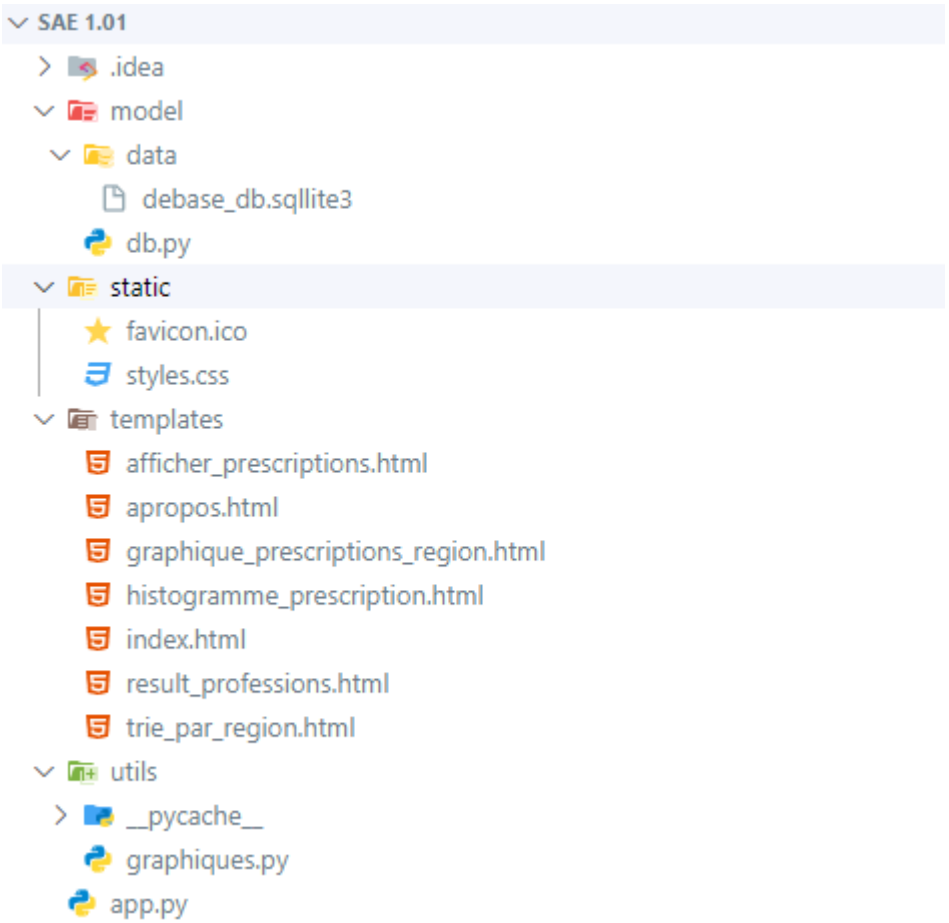
```
    <!-- Lien vers le fichier CSS -->
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
    <!-- Icone du site -->
    <link rel="icon" href="{{ url_for('static', filename='favicon.ico') }}" type="image/png">
```

Enfin dans il reste un dossier « utils » contenant le fichier « graphiques.py » où se trouve différentes fonctions où la bibliothèque Matplotlib est utilisée.

Afin de relier tous ces fichiers entre eux, de gérer les routes Flask pour les pages HTML, de connecter les différentes requêtes avec l'application web il reste le fichier dit « contrôleur » app.py. Cette architecture suit une structure MVC (Modèle : db.py, Vue : pages HTML, Contrôleur : app.py). Par exemple dans le fichier app.py, voici une fonction qui gère un graphique crée avec la requête SQLite qui est associée et la page HTML ou le graphique est créé :

```python
@app.route('/histogramme_prescription', methods=['GET', 'POST'])
def histogramme_prescription():
    annee = request.form.get('annee', type=int)
    if not annee:
        annee = None
    histogram_image = graphique(annee)
    return render_template('histogramme_prescription.html', histogram_image=histogram_image)
```

Voici l'architecture finale de notre projet :



5/Implémentation des fonctionnalités :

Consultation des données :

Le projet intègre plusieurs fonctionnalités, allant de la consultation des données à leur analyse statistique et leur visualisation graphique. Chaque fonctionnalité repose sur

une implémentation en backend via les fichiers db.py et app.py et la partie frontend (l'interface web).

Pour la partie backend on crée d'abord une fonction dans le fichier

db.py :

```
def montant_moyen_prescriptions(annee=None):
    conn = connect_db()
    query = """
    SELECT libelle_poste_prescription, AVG(montant_moyen_prescription_integer) AS montant_moyen, Annee
    FROM Prescriptions
    """
    if annee:
        query += f" WHERE Annee = {annee}"

    query += " GROUP BY libelle_poste_prescription;"

    result = pd.read_sql_query(query, conn)
    conn.close()
    return result
```

Puis, on intègre cette fonction dans le fichier app.py, voici un extrait de la fonction associée à la requête ci-dessus :

```
@app.route('/afficher_prescriptions', methods=['GET'])
def afficher_prescriptions():
    annee_min = request.args.get('annee_min', type=int)
    annee_max = request.args.get('annee_max', type=int)
    search_query = request.args.get('search', '').lower()
    order_annee = request.args.get('order_annee', default='asc', type=str)

    Prescriptions = db.visualiser_table()

    Prescriptions['Id_Prescription'] = Prescriptions['Id_Prescription'].astype(int)
    Prescriptions['Annee'] = Prescriptions['Annee'].astype(int)

    # Filtre par années si les paramètres sont renseignés
    if annee_min:
        Prescriptions = Prescriptions[Prescriptions['Annee'] >= annee_min]
    if annee_max:
        Prescriptions = Prescriptions[Prescriptions['Annee'] <= annee_max]
```

Enfin, on crée l'interface web qui génère le résultat des deux fonctions ci-dessus grâce au code Flask intégré dans le fichier HTML de la page.

6/Interface utilisateur :

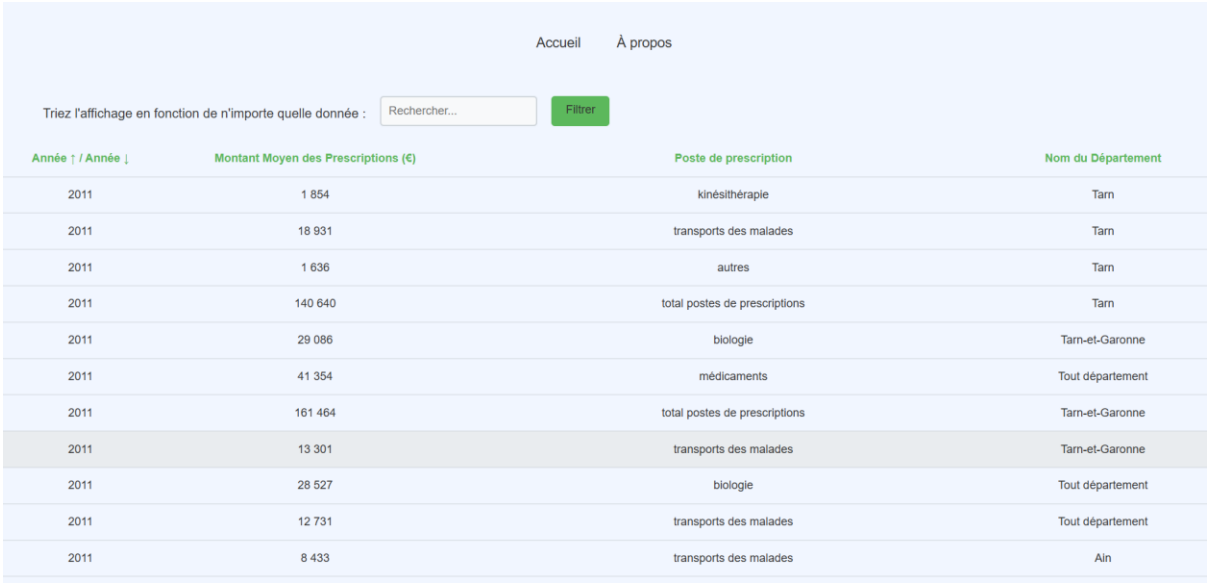
L'interface utilisateur propose une navigation fluide entre différentes pages, facilitant l'accès aux principales fonctionnalités. La page d'accueil présente un accueil clair avec des boutons redirigeant vers les différentes analyses possibles : Prescriptions, Histogramme, Trie par région et Graphique Régions. Des menus de navigation sont présents sur toutes les pages, permettant un accès rapide aux différentes sections, y compris une page "À propos".

Chaque page HTML offre une fonctionnalité spécifique :

- **index.html** : Page principale accueillant l'utilisateur avec des options de navigation.

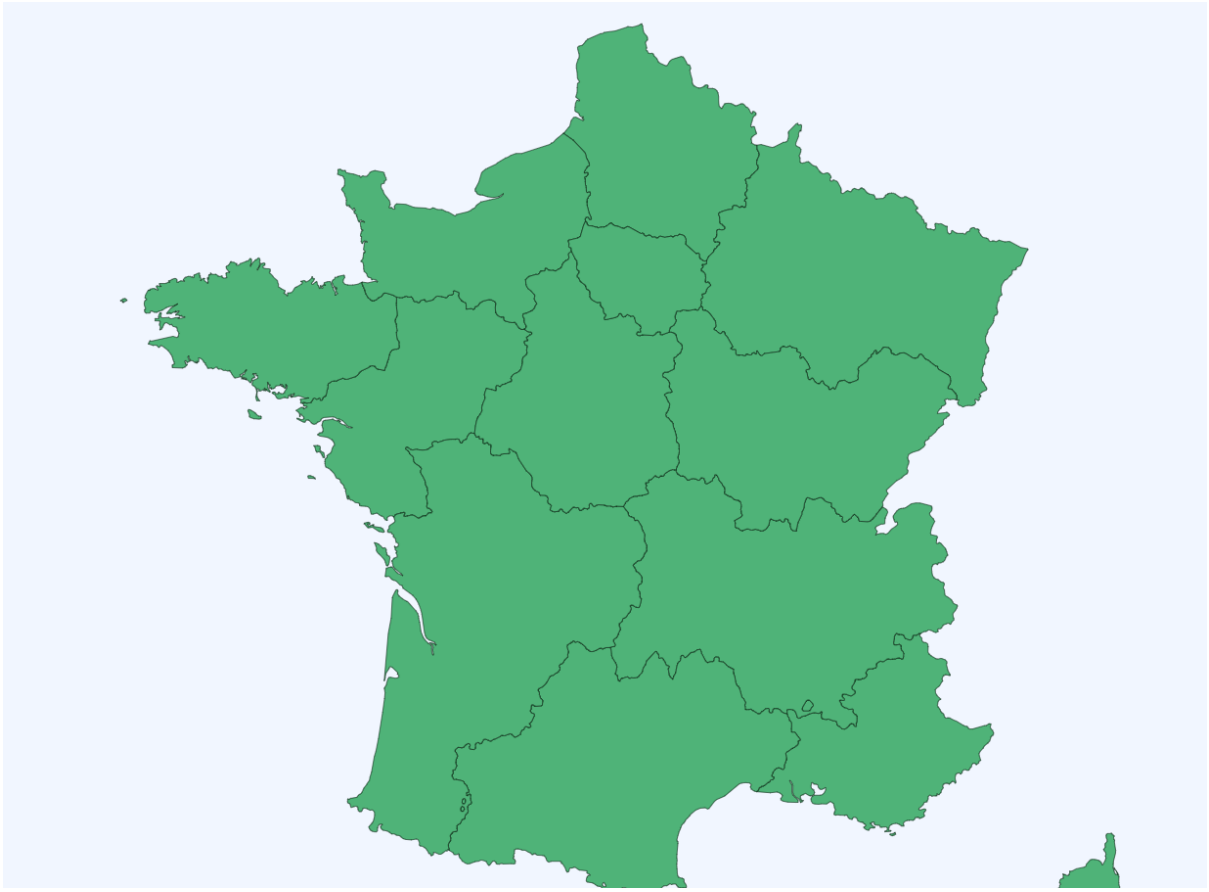


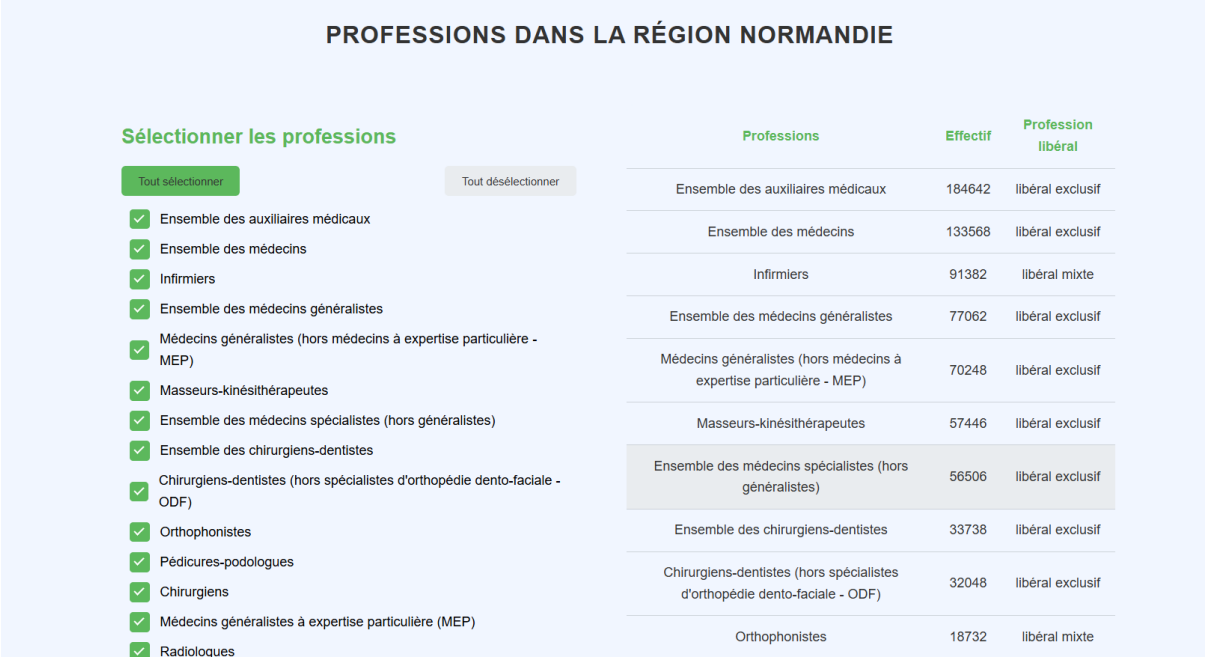
- **afficher_prescriptions.html** : Affichage des prescriptions sous forme de tableau interactif avec filtres et tri.



Année ↑ / Année ↓	Montant Moyen des Prescriptions (€)	Poste de prescription	Nom du Département
2011	1 854	kinésithérapie	Tarn
2011	18 931	transports des malades	Tarn
2011	1 636	autres	Tarn
2011	140 640	total postes de prescriptions	Tarn
2011	29 086	biologie	Tarn-et-Garonne
2011	41 354	médicaments	Tout département
2011	161 464	total postes de prescriptions	Tarn-et-Garonne
2011	13 301	transports des malades	Tarn-et-Garonne
2011	28 527	biologie	Tout département
2011	12 731	transports des malades	Tout département
2011	8 433	transports des malades	Ain

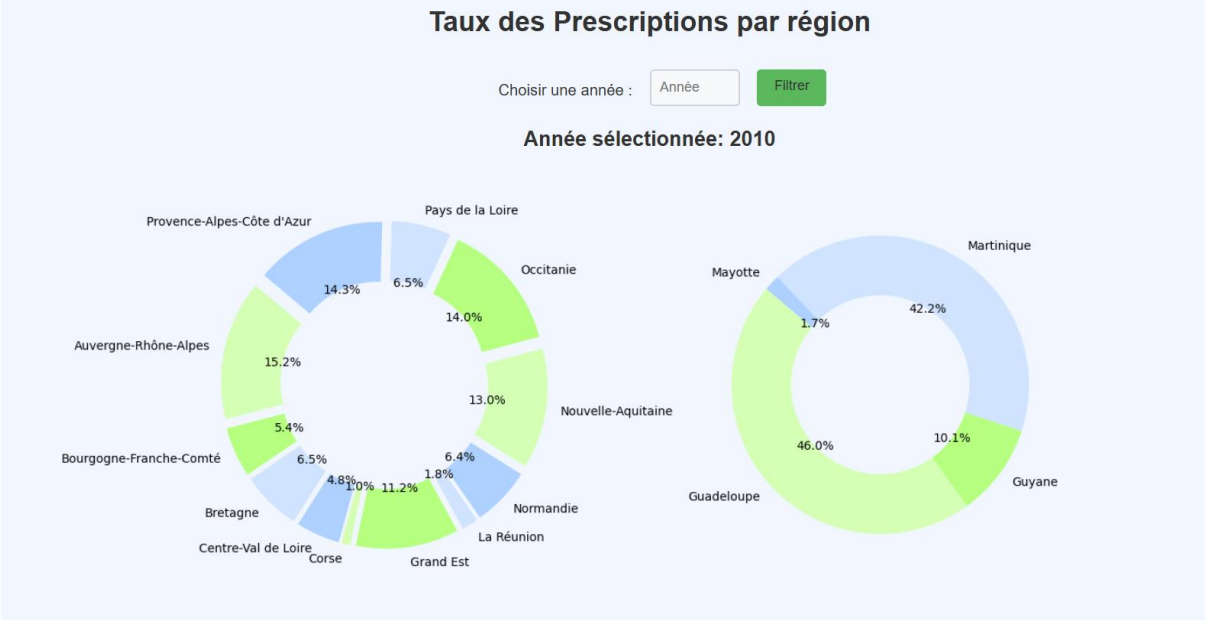
- **trie_par_region.html** : Carte interactive affichant les prescriptions par région.



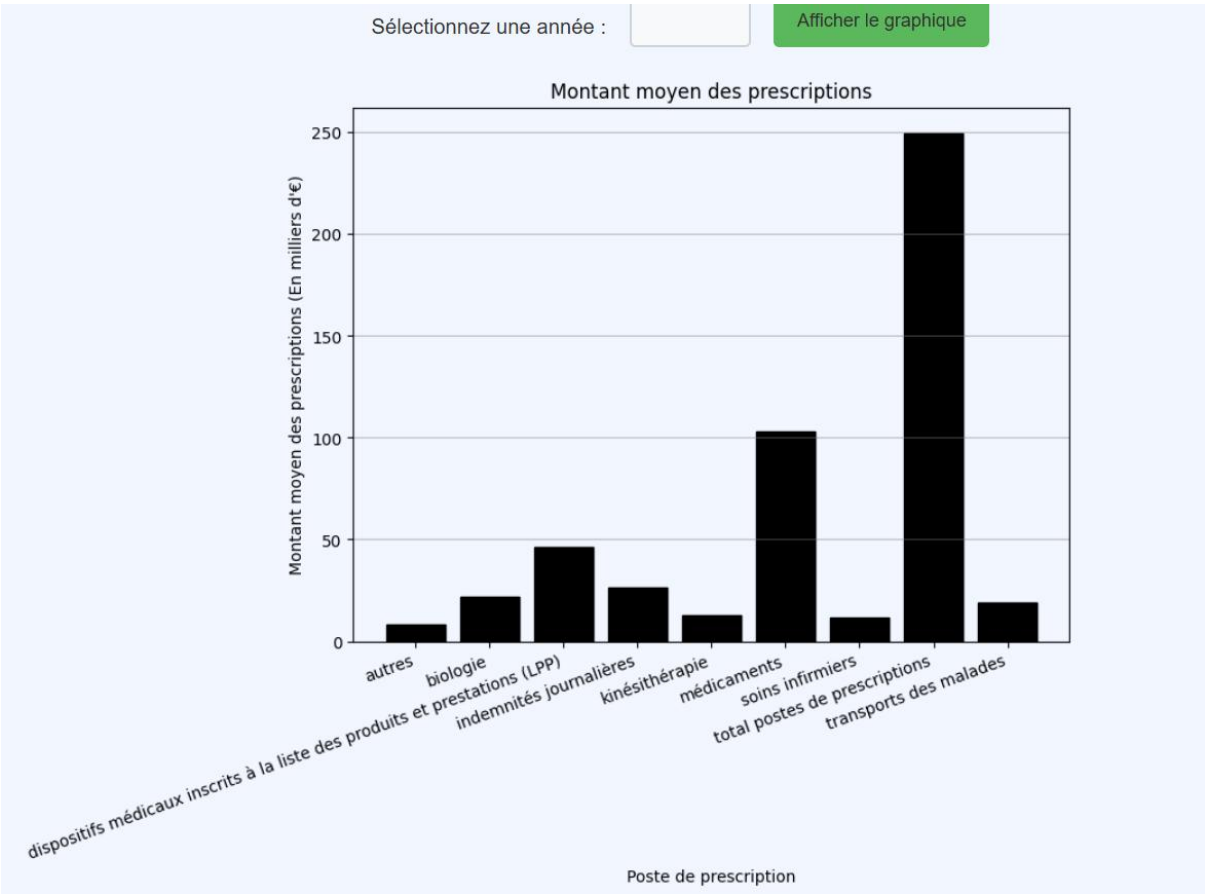


Professions	Effectif	Profession libéral
Ensemble des auxiliaires médicaux	184642	libéral exclusif
Ensemble des médecins	133568	libéral exclusif
Infirmiers	91382	libéral mixte
Ensemble des médecins généralistes	77062	libéral exclusif
Médecins généralistes (hors médecins à expertise particulière - MEP)	70248	libéral exclusif
Masseurs-kinésithérapeutes	57446	libéral exclusif
Ensemble des médecins spécialistes (hors généralistes)	56506	libéral exclusif
Ensemble des chirurgiens-dentistes	33738	libéral exclusif
Chirurgiens-dentistes (hors spécialistes d'orthopédie dento-faciale - ODF)	32048	libéral exclusif
Orthophonistes	18732	libéral mixte

- **graphique_prescriptions_region.html** : Visualisation graphique des prescriptions par région.



- **Montant moyen des prescriptions** : Histogramme affichant les montants des prescriptions selon chaque prescription.



L'utilisation de CSS permet d'obtenir une interface claire et bien structurée, avec des couleurs et des espacements appropriés. De plus, un responsive design a été mis en place afin d'adapter l'interface aux différentes tailles d'écran et améliorer ainsi l'accessibilité.

7/Tests et Validation des Fonctionnalités

Des tests unitaires ont été réalisés pour vérifier les fonctions Python utilisées dans le traitement des données. De plus, des tests d'interface ont été effectués pour valider la navigation, l'affichage des graphiques et le bon fonctionnement des filtres. Des tests spécifiques ont également été menés pour vérifier les résultats retournés lors de la recherche et du tri des données.

Résultats des Tests

- **Cas d'utilisation testés :**
    - Affichage des prescriptions par année et par poste de prescription : fonctionnement correct.

    - Filtres et tri appliqués aux tableaux de données : résultats conformes aux attentes.
    - Affichage des cartes interactives et graphiques : données bien représentées.

- **Problèmes rencontrés et solutions :**
    - Erreur d'affichages de données sur le site web
    - Modification de graphique camembert car pourcentage biaisé a cause de l'attribut tout départements

8/Résultats et Analyse

Les données analysées proviennent des prescriptions médicales enregistrées par région et par année. Les principaux indicateurs obtenus sont :

- Montant moyen des prescriptions en euros.
- Nombre de prescriptions par poste et par département.

L'analyse des données permet de dégager des tendances significatives :

- **Disparités régionales :** Certaines régions présentent des montants de prescriptions plus élevés que d'autres.
- **Évolution temporelle :** Les montants moyens des prescriptions montrent des variations au fil des ans, nécessitant une analyse approfondie des facteurs sous-jacents.

- **Carte des prescriptions par région :** Met en évidence les zones où les prescriptions sont les plus importantes.
- **Histogrammes des montants moyens :** Permet d'identifier les évolutions et les postes de prescription les plus coûteux.
- **Analyse des tendances :** Mise en lumière des hausses et baisses des prescriptions en fonction des périodes et des régions.

Conclusion

Conclusion

Ce projet a permis de développer une application web interactive permettant l'analyse et la visualisation des données de l'Assurance Maladie. Grâce à l'utilisation d'une base de données SQLite, d'un backend en Flask et d'un frontend structuré en HTML/CSS,

nous avons pu concevoir un outil efficace et intuitif pour explorer les prescriptions médicales sous différentes formes (tableaux, graphiques, cartes interactives).

L'organisation du travail en équipe a joué un rôle clé dans la réussite du projet. La répartition des tâches entre le développement de l'interface utilisateur et l'intégration des bases de données a permis d'assurer un avancement fluide et structuré. L'utilisation de bibliothèques comme Pandas et Matplotlib a facilité le traitement et la représentation visuelle des données, rendant l'analyse plus accessible et pertinente.

Les tests réalisés ont permis d'identifier et de corriger plusieurs problèmes, notamment des erreurs d'affichage et des biais dans certaines représentations graphiques. Ces ajustements ont contribué à améliorer la performance et la précision des résultats affichés sur l'application.

En conclusion, notre projet répond aux objectifs initiaux en offrant une plateforme claire et fonctionnelle pour l'exploration des données médicales. Il met en lumière des tendances intéressantes, comme les disparités régionales ou l'évolution des prescriptions au fil des ans. Ce travail pourrait être enrichi par l'ajout de nouvelles fonctionnalités, telles que des analyses prédictives ou des comparaisons plus approfondies entre les différentes professions médicales.